

FINAL-YEAR PROJECT · JUNE 2026

Toxic & hate-speech comment detection

The modelling journey behind Raven — from classical and RNN baselines, to a fine-tuned transformer, to ensemble learning by simple averaging — and our own original method for Hindi, **RAVEN-X**.

Every number in this document was read from our own code and notebook outputs, or from a cited paper. Where a step is reasoning or a plan rather than a result we ran, we say so.

0.9515

DistilBERT ROC-AUC,
balanced 20k val

MEASURED**0.919**

best RNN bias-aware AUC
(BiGRU-Conv1D+Attn)

MEASURED**0.842**

RAVEN-X MuRIL macro-F1,
HASOC Hindi

MEASURED**Author:** Niladri Hazra & team**Supervisor:** Dr. Arpita Dutta

CONTENTS

What this document covers

We walk through the project once, then through every model we tried in order — what it did, why it falls short (both the maths and what we actually saw), and how the next step fixes it — ending with our own method and the full architecture diagrams.

- 01** What this project is

- 02** Data and how we measure

- 03** Stage 1 — Classical & RNN models (LSTM family)

- 04** Stage 2 — DistilBERT: a fine-tuned transformer

- 05** Stage 3 — Scaling up to a larger transformer

- 06** Stage 4 — Ensemble learning by simple averaging

- 07** The final model — what it does and where it stops

- 08** RAVEN-X — our original Hindi method

- 09** What we need next

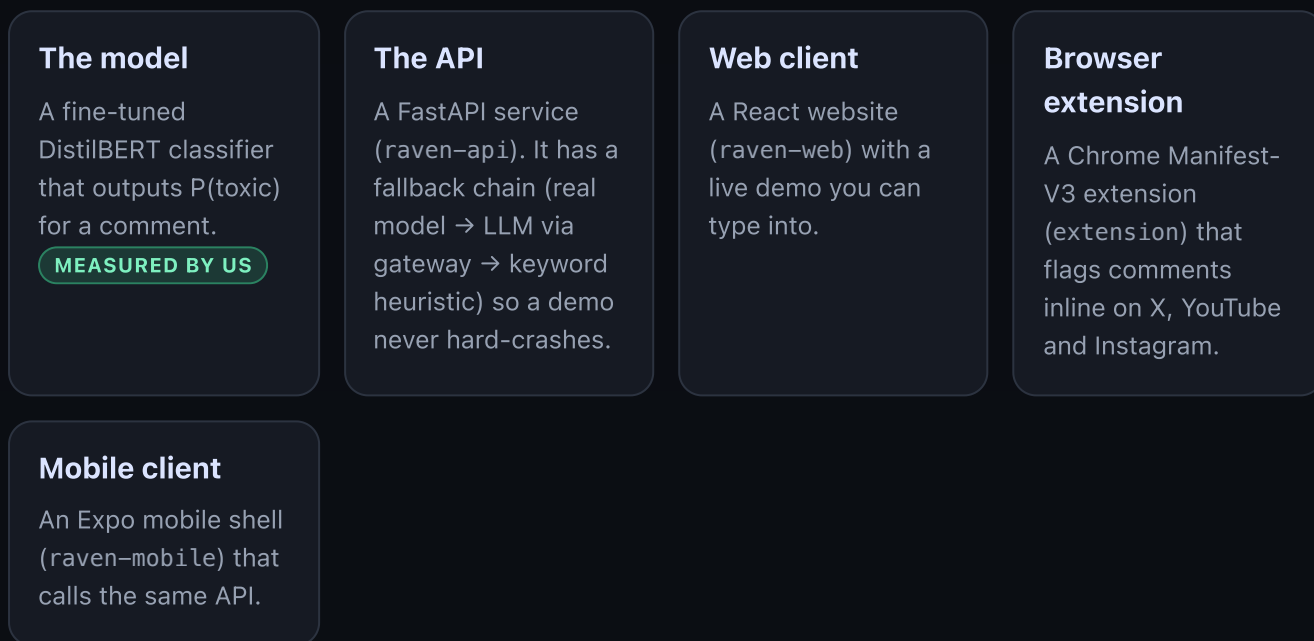
- 10** References

What this project is

Raven is a system that detects toxic and hate-speech comments. It answers one question: **which comments need a human moderator to look at?** Instead of a person reading every comment, Raven flags the risky ones as "needs review" and lets the safe ones through. The point is to save the moderator's time, not to replace their judgement.

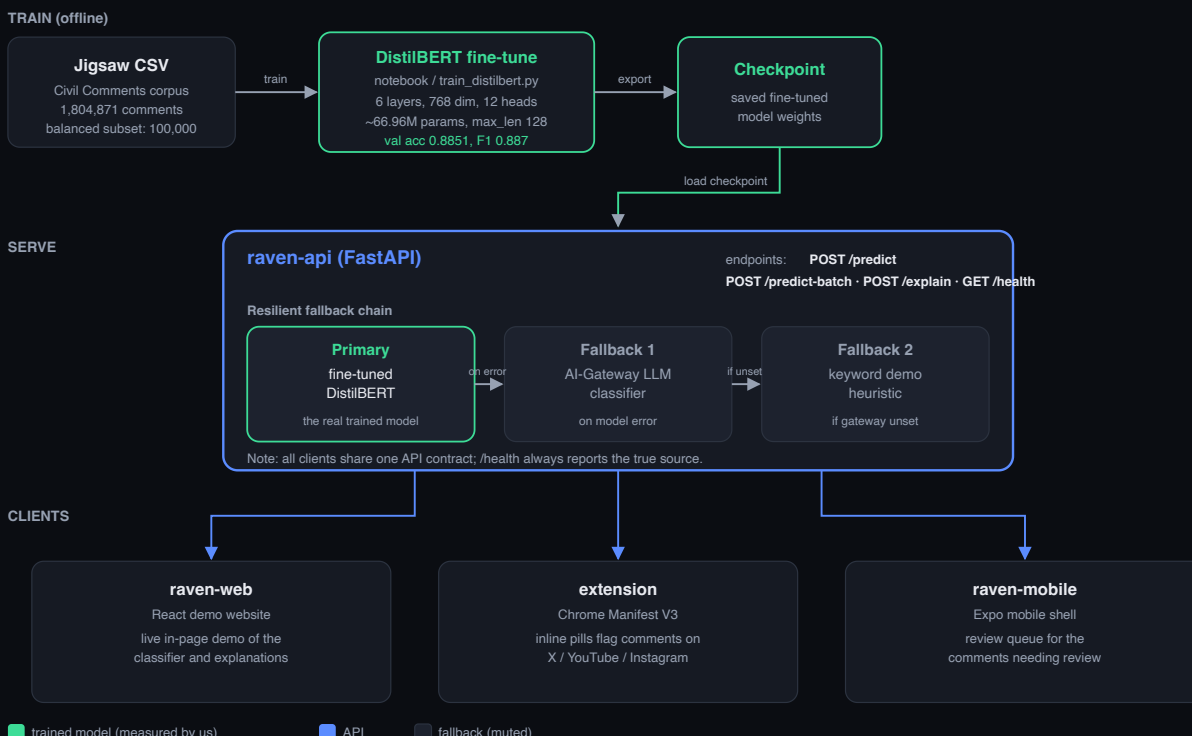
What we built

One model, one API, and three clients. The three clients all speak the same API contract.



Raven system architecture: one model, one API, three clients

Triages comments so a human moderator reads only the risky ones. English DistilBERT track.



■ trained model (measured by us) ■ API ■ fallback (muted)

Figure 1 — end-to-end system: training to three clients.

The dataset (English track)

We trained the English model on the Jigsaw "Unintended Bias in Toxicity Classification" corpus (Civil Comments).

Property	Value
Total comments	1,804,871 MEASURED BY US
Class balance	1,660,537 non-toxic vs 144,334 toxic (~11.5 : 1, about 8% toxic)
Label	continuous target in [0,1], binarised at 0.5 (target $\geq 0.5 \rightarrow$ toxic)
Bias measurement	identity columns (muslim, black, white, female, ...) allow per-subgroup checks

Honest caveat. Two model families in this document were measured under two different evaluation regimes, and we keep them separate. The RNN/classical family was evaluated on the large split with the Jigsaw bias-aware AUC. DistilBERT was evaluated on a balanced 100,000-comment down-sample (50,000 per class) with plain accuracy/F1/ROC-AUC. The two sets of numbers are **not** directly comparable.

Roadmap — how this document reads

The talk walks up a ladder of model capacity, and then turns to our own contribution.

1. **Classical / TF-IDF** — bag-of-words with logistic regression, the simplest baseline.
2. **LSTM & RNN family** — single LSTM, BiLSTM, weighted BiLSTM, BiGRU with attention.
3. **DistilBERT** — our transformer: a distilled backbone we fine-tuned.
4. **Larger transformer** — the next capacity step (reasoning + literature, **PROPOSED NEXT STEP**, not run by us).
5. **Ensemble** — simple averaging of diverse models (theory + Kaggle literature).
6. **Final model** — what it can and cannot do, stated honestly.
7. **RAVEN-X** — our original contribution: cross-script rationale transfer for low-resource Hindi.
8. **Architecture diagrams** — how the pieces fit together.

Takeaway. Raven is a toxicity triage system: one fine-tuned model behind one API, used by three clients. This document climbs from a simple baseline up to a transformer and an ensemble, then presents RAVEN-X, our own method for low-resource Hindi.

Data and how we measure

The English track uses the **Jigsaw "Unintended Bias in Toxicity Classification"** data (the Civil Comments corpus). It has **1,804,871** comments **MEASURED BY US**. Each comment carries a continuous target in $[0,1]$. We treat a comment as toxic when target ≥ 0.5 , so the score is binarised at 0.5. The data also has identity columns (muslim, black, white, female, and others) that let us measure bias by group.

Why balancing and the right metric matter

The raw data is heavily imbalanced: **1,660,537** non-toxic against **144,334** toxic comments **MEASURED BY US**. That is about **11.5 : 1**, roughly 8% toxic. On data like this, plain accuracy is misleading. A model that labels everything "non-toxic" already scores about 92% accuracy while catching no toxic comment at all. So we cannot trust accuracy alone; we have to look at how the minority (toxic) class is handled.

The metrics, one line each

Metric	Plain definition
Accuracy	Fraction of all comments labelled correctly.
Precision (toxic)	Of the comments we flagged toxic, how many really were.
Recall (toxic)	Of the truly toxic comments, how many we caught.
F1 (toxic)	Harmonic mean of precision and recall — one number that balances both.
Macro-F1	F1 computed per class, then averaged — treats both classes equally regardless of size.
ROC-AUC	Probability the model ranks a random toxic comment above a random non-toxic one.
Jigsaw bias-aware AUC	A weighted blend of the overall AUC and per-identity-subgroup AUCs, so a model is rewarded only if it ranks well within each identity group, not just on average.

Two evaluation regimes — kept separate on purpose

The numbers in this project come from two different setups. We keep them apart because mixing them would be a mistake.

Regime A — RNN / classical family

- Trained and evaluated on the large, imbalanced split.
- Reported with the **Jigsaw bias-aware AUC**.
- Keras tokenizer, vocab **100,000**, `max_sequence_length 300`.
- Pretrained 300-d word embeddings, shape **(276,289 × 300)**.

Regime B — DistilBERT

- Trained on a **balanced down-sample: 50,000 per class = 100,000**.
- Split **80/20 → 80,000 train / 20,000 val** (10k per class).
- Reported with plain accuracy / F1 / ROC-AUC on that balanced val set.
- WordPiece tokenizer, 30,522 vocab, `max_length 128`.

Honest caveat. The RNN/classical numbers (Regime A) and the DistilBERT numbers (Regime B) were measured under different data splits, sequence lengths, and metrics — one uses bias-aware AUC on the large imbalanced set, the other uses plain ROC-AUC on a balanced 20k val set. So their headline numbers are **not directly comparable**. We do not put them side by side and pick a winner on raw accuracy. Instead we compare **behaviour**, especially recall on the toxic class.

Takeaway. For moderation, recall on the toxic (minority) class is the number that actually matters. A missed toxic comment (false negative) is the costly error — it reaches users. High accuracy on an 11.5:1 split can hide a model that misses most toxic comments, so we judge models by how much of the toxic class they catch.

Stage 1 — Classical and RNN models (the LSTM family)

This was the first modelling stage on the English Jigsaw task. The aim was to set a baseline before trying anything heavier. We built a bag-of-words classifier and then a family of recurrent networks, all on the same data.

Honest caveat. These classical and RNN baselines were primarily a teammate's contribution (Debangshu Dey). We use them here as a comparison family. The DistilBERT transformer and the product stack are the author's (Niladri Hazra) track. We do not claim the author trained the LSTMs.

(a) What we built

All RNN models used the same input setup: a Keras tokenizer with a vocabulary of 100,000, sequences padded or cut to a maximum length of 300, and pretrained word embeddings of shape $276,289 \times 300$. These embeddings are **context-free**: each word has one fixed 300-d vector, the same vector no matter where the word appears.

- TF-IDF + Logistic Regression — a bag-of-words classifier (SGD, log-loss, $\alpha = 1e-5$).
- Single LSTM (128 units, 10 epochs).
- BiLSTM — two BiLSTM layers of 128 units, global max pooling, dense head.
- Weighted BiLSTM — the same with class weights to fight imbalance.
- BiGRU + Attention (64 units, attention).
- BiGRU-Conv1D + Attention (64 units, attention, plus a Conv1D layer).

(b) Real results

These are measured numbers **MEASURED BY US**. "Bias-AUC" is the Jigsaw bias-aware AUC (a weighted blend of overall AUC and per-identity-subgroup AUCs). "Val acc" is the best validation accuracy.

Model	Architecture detail	Val acc	Bias-AUC
TF-IDF + Logistic Regression	bag-of-words (SGD, log-loss, $\alpha = 1e-5$)	0.87	0.9061
Single LSTM	LSTM 128u, 10 epochs, pretrained 300-d emb	0.9469	0.8927
BiLSTM	2× BiLSTM 128u + GlobalMaxPool + Dense	0.8114	0.8992
Weighted BiLSTM	same + class weights	0.8561	0.9056
BiGRU + Attention	BiGRU 64u + attention	0.9517	0.9181
BiGRU-Conv1D + Attention	BiGRU 64u + attention + Conv1D	0.9514	0.9190

Classical & RNN baselines — Jigsaw bias-aware AUC

Higher is better, y-axis zoomed to 0.88–0.92 so the small gaps are visible. Regime A; measured by us.

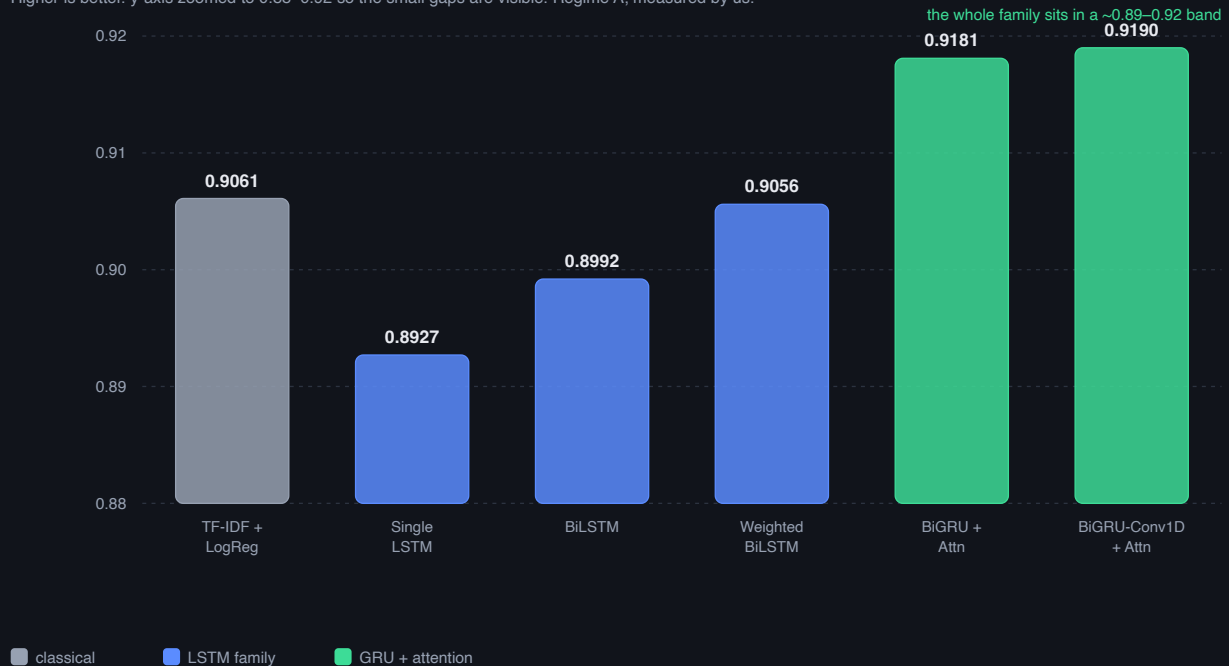


Figure 2 — Bias-aware AUC across the six classical and RNN baselines (measured by us).

(c) The mathematical reason the RNN approach is limited

The limits below are general theory [LITERATURE](#), not specific to our run.

$$h_t = f(h_{t-1}, x_t)$$

path length between token i and token $j = O(|i - j|)$ steps

→ a signal from token i to token j passes through $|i - j|$ multiplications

→ repeated products shrink or grow → vanishing / exploding gradients

- **Sequential recurrence.** The hidden state is computed step by step. The path between two tokens grows with their distance, so a long-range dependency must survive many matrix multiplications. LSTM gating (Hochreiter & Schmidhuber, 1997) softens this but does not remove it. [LITERATURE](#)
- **No parallelism over the sequence.** Because step t needs step $t-1$, the network must unroll one token at a time, so training and inference are slow. [LITERATURE](#)
- **Context-free embeddings.** One fixed vector per word. "kill" in "you killed it!" (praise) and "kill" in a threat start from the same vector, so the RNN must separate them from a weak signal. [LITERATURE](#)
- **Information bottleneck.** The whole left context is squeezed into a single fixed-size hidden vector. [LITERATURE](#)

(d) The drawback we actually saw

These are our measured numbers [MEASURED BY US](#).

- **High accuracy hides low toxic recall.** On the imbalanced set, TF-IDF+LogReg has a toxic recall of about 0.448 and the Single LSTM about 0.452. They miss more than half of toxic comments while still reading as roughly "94% accurate" on imbalanced data. BiLSTM reaches a toxic recall of about 0.914, but at a much lower validation accuracy (0.8114).
- **Diminishing returns from architecture tweaks.** Making the model bidirectional, then weighted, then attention-augmented only moves the Bias-AUC from about 0.89 up to about 0.92. The extra machinery does not break past this band.

Takeaway. The ceiling here is the representation, not the wiring. Context-free word vectors plus a sequential, fixed-size bottleneck cap what these models can learn, and stacking more recurrent layers on top of the same embeddings gives little back. This is what motivates a contextual, attention-based, pretrained transformer.

(e) What it motivates

The next stage replaces context-free vectors with context-dependent ones and replaces the sequential bottleneck with attention, so every token can reach every other token in one step. That is Stage 2: fine-tuning a pretrained transformer (DistilBERT).

Stage 2 — DistilBERT: a fine-tuned transformer

The RNN models in Stage 1 read a sentence one word at a time and kept a single fixed vector per word. That gave two problems: long-range context decays as it passes through the chain, and a word like "killed" gets the same vector whether it is a threat or a sports headline. A transformer fixes both. We did not train one from scratch — we took an existing pretrained model and fine-tuned it.

Why attention beats recurrence

The transformer (Vaswani et al., 2017 [LITERATURE](#)) replaces recurrence with self-attention. Three properties matter here:

- **Short path length.** Self-attention connects every pair of tokens directly, so the path between any two words is **$O(1)$** , against **$O(n)$** for an RNN that must pass information step by step. Long-range context is reached in one hop. [LITERATURE](#)
- **Contextual, bidirectional embeddings.** Each token's vector is computed from the whole sentence, so the same word gets a different representation in a different context. This is what removes the fixed-vector limit behind the "killed it" problem. [LITERATURE](#)
- **Parallel and pretrained.** All positions are processed in parallel (no sequential dependency), and the backbone is pretrained on a large corpus. Fine-tuning then needs only a little task data — this is transfer learning. [LITERATURE](#)

RNN inter-token path = $O(n)$ → signal decays over distance
 self-attention inter-token path = $O(1)$ → direct long-range link

What we actually did

We did **not** distill anything ourselves. We took the already-distilled `distilbert-base-uncased` and fine-tuned it. DistilBERT (Sanh et al., 2019 [LITERATURE](#)) is the result of distilling BERT-base into a smaller student: about **66M parameters**, **6 transformer layers**, roughly **40% smaller** and **60% faster** than BERT while keeping about **97%** of its performance [LITERATURE](#). We fine-tuned it on a balanced 100k subset of Jigsaw (50k toxic + 50k non-toxic, split 80k train / 20k val)

[MEASURED BY US](#).

The architecture, layer by layer: WordPiece tokenizer (vocab **30,522**, max length **128**) → embeddings → **6 transformer blocks** (each with **12 attention heads** and a feed-forward block $768 \rightarrow 3072 \rightarrow 768$) → the first-token **[CLS]** hidden state (768-d) → pre_classifier Linear(768→768) with ReLU and dropout → classifier Linear(768→2) → softmax giving $P(\text{toxic})$. Training used the HuggingFace Trainer with AdamW, learning rate about $5e-5$, batch size 16, on a single GPU.

DistilBERT for sequence classification

Top-to-bottom data flow. Parameter counts are shown on the right of each block.

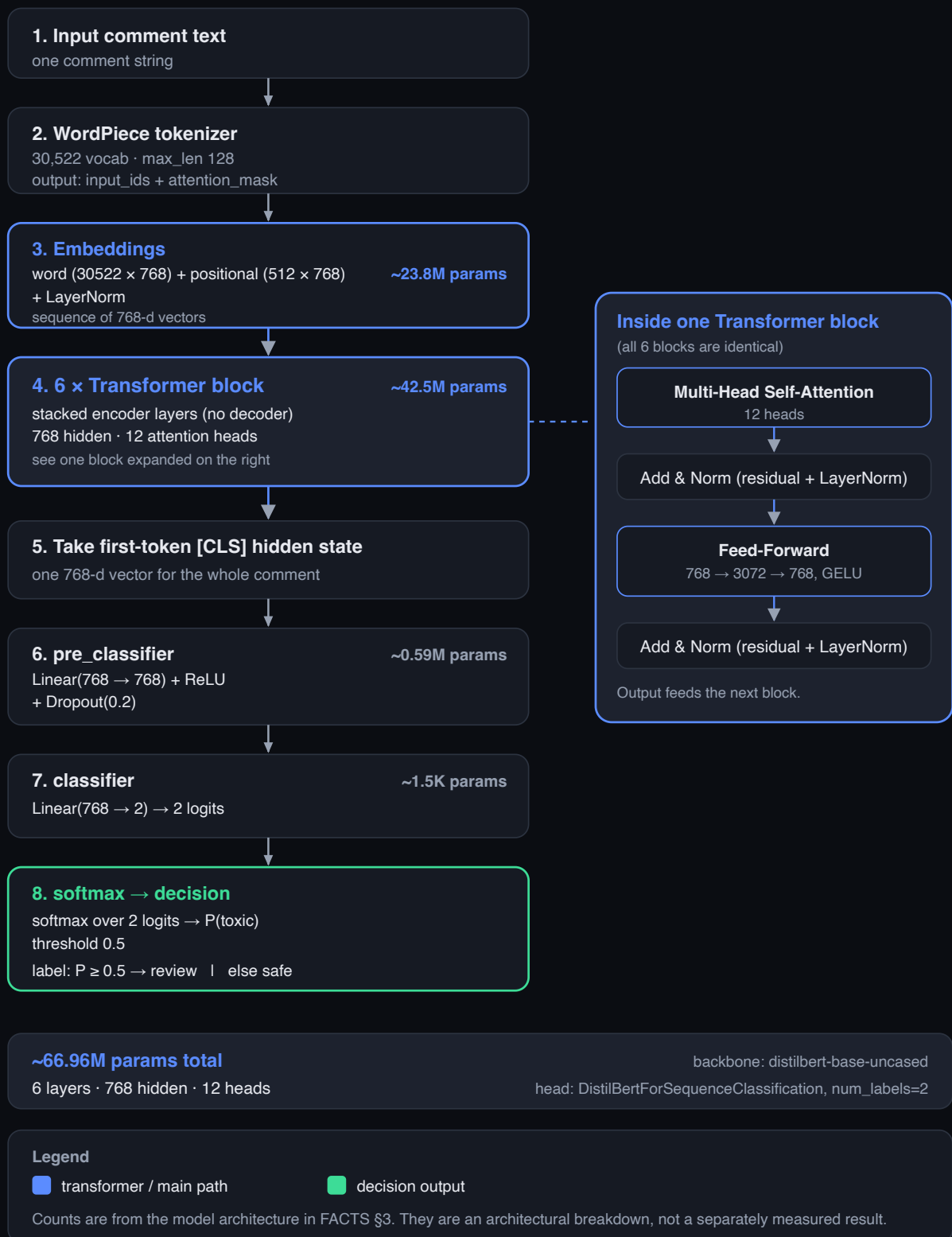


Figure 3 — DistilBERT-for-sequence-classification, layer by layer (with parameter counts).

Our real results

All numbers below are on the balanced 20k validation set (10k per class) **MEASURED BY US**. We report two runs: 1 epoch (2,500 steps) and the saved 3-epoch model (7,500 steps).

Metric (balanced 20k val)	1 epoch	3 epochs (saved final)
Accuracy	0.8895	0.8851
Precision (toxic)	0.873	0.875
Recall (toxic)	0.912	0.899
F1 (toxic)	0.892	0.887
Validation loss	0.537	0.818
Training loss	0.6075	0.4207
ROC-AUC	not reported	0.9515
PR-AUC (avg precision)	not reported	0.9495

Confusion matrix for the 3-epoch model (rows = actual, columns = predicted; 10k per class)

MEASURED BY US:

	Pred non-toxic	Pred toxic
Actual non-toxic	TN 8712	FP 1288
Actual toxic	FN 1011	TP 8989

Takeaway. The win over the RNN family is in behaviour, not raw accuracy. DistilBERT reaches about **0.90 toxic recall** on the balanced 20k val set **MEASURED BY US**, while the simple high-accuracy baseline TF-IDF + Logistic Regression had toxic recall below **0.45** on the large imbalanced split **MEASURED BY US** — it missed more than half of all toxic comments. Both DistilBERT runs (1 epoch and 3 epochs) are Regime B (balanced 20k val); the TF-IDF + Logistic Regression recall above is Regime A (the large imbalanced split). Recall on a balanced set and recall on a heavily imbalanced set are different regimes, so this is not a like-for-like number — treat the DistilBERT-vs-RNN-family comparison as a qualitative illustration of the imbalance-failure point, not a direct metric.

The drawback that remains

Honest caveat. Four limits stay with this model. **(1) It is a compressed model** — 66M params, 6 layers — so it has a capacity ceiling. **(2) Mild overfitting at 3 epochs:** training to 3 epochs lowered training loss (0.6075 → 0.4207) but raised validation loss (**0.537 → 0.818**) with no accuracy gain, so about 1 epoch was near-optimal for this 100k subset. **(3) It is a single model** — its errors have an irreducible variance that one model cannot average away. **(4) Scope limits:** English, uncased, a 128-token cap, and trained only on the balanced subset.

This points to two ways forward. Raise the capacity ceiling by scaling the model up (Stage 3), and reduce single-model variance by combining several models (Stage 4).

Stage 3 — Scaling up to a larger transformer

The next idea was to ask: what if we made the model bigger? In an early discussion this was phrased as moving to a "DistilBERT large". We want to be honest about two things up front.

Honest caveat. We did **not** run any larger-model experiment. There is no measured number on this slide. Everything here is reasoning plus what other people have reported. Treat every claim as

LITERATURE or **PROPOSED NEXT STEP**, never as something we measured.

First, the naming

There is no official "DistilBERT-large". DistilBERT only ships as a base model (~66M params, 6 layers). DistilBERT is itself a *compressed* version of BERT-base. So the real way to add capacity is to **undo the compression** and use a full-size encoder instead:

Encoder	Params	Layers	Heads	Hidden dim	Status
DistilBERT-base (what we used)	~66.96M	6	12	768	MEASURED BY US
BERT-base	110M	12	12	768	PROPOSED NEXT STEP
BERT-large	~340M	24	16	1024	PROPOSED NEXT STEP

Why a bigger model should help — bias-variance reasoning

A model's expected error can be split into three parts. This is standard theory, not a measurement.

Expected error = $\text{bias}^2 + \text{variance} + \text{irreducible noise}$

bias^2 → how far the model class is from the true function (too simple = high bias)

variance → how much the fit changes with a different training sample

noise → label / data randomness we cannot remove

A larger encoder has more capacity, so it can fit the true function more closely. That lowers **bias**. This is the reason fine-tuned transformers were the best *single* models in the Jigsaw competition

LITERATURE.

What we gain

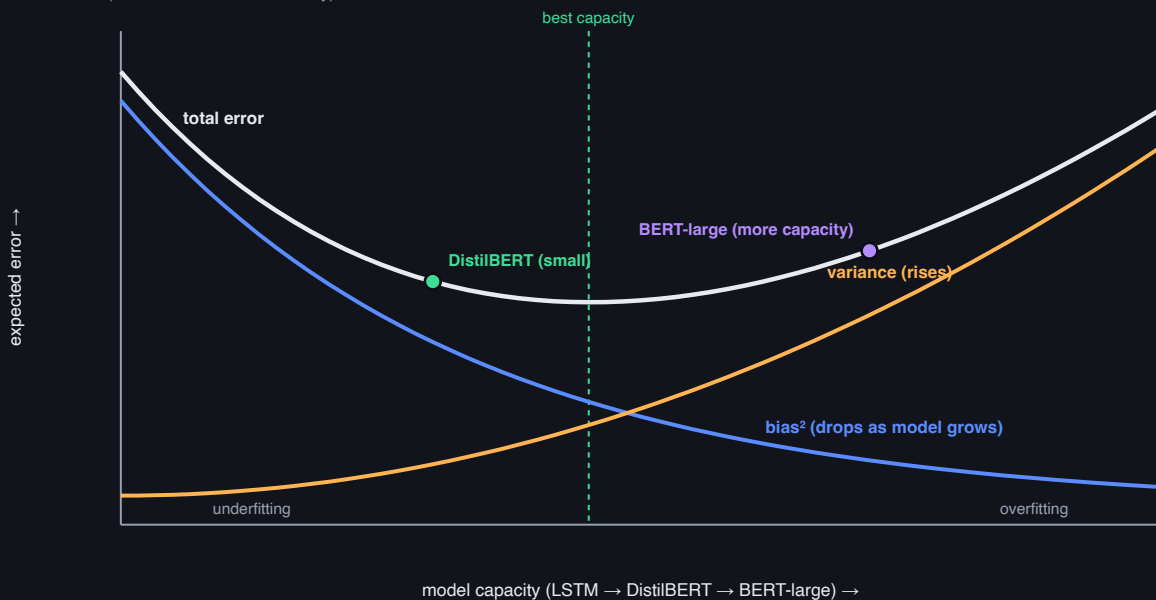
- Lower bias — more capacity to fit the true function.
- Best single-model behaviour, matching the Jigsaw result **LITERATURE**.

What we pay

- Higher variance / overfitting risk on a small balanced set (ours was 100,000 rows).
- Much more compute and memory (BERT-large is ~340M vs ~66.96M).
- Diminishing returns — each size step buys less.

Why a bigger model is not automatically better

Illustrative (standard bias-variance theory) — not our measurements.



A single larger encoder moves right: it lowers bias but adds variance — so it raises the ceiling, it does not remove single-model error.

Figure 4 — Bias–variance trade-off: a bigger encoder lowers bias but raises variance (illustrative).

We have a real reason to worry about the variance side. Our 3-epoch DistilBERT run already showed mild overfitting: training loss fell (0.6075 → 0.4207) while validation loss rose (0.537 → 0.818) with no accuracy gain **MEASURED BY US**. A larger model on the same small set would likely overfit sooner, not later.

Honest punchline. A single bigger model raises the ceiling, but it does **not** remove single-model variance or correlated errors — one model still makes the same kind of mistakes every time. In the Jigsaw competition no top team relied on one model; the best teams averaged several diverse models **LITERATURE**. That is what Stage 4 looks at.

Takeaway. Going bigger (BERT-base 110M or BERT-large ~340M) is the honest way to add capacity, and it would lower bias — but it costs far more compute and does not fix the variance problem. So the more useful next step is not "one bigger model", it is "several diverse models averaged together".

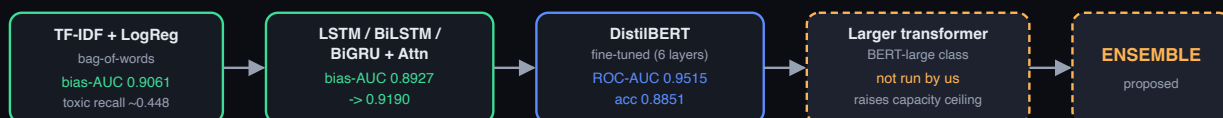
Stage 4 — Ensemble learning by simple averaging

This is the technique we are bringing in now. The idea is short: train several **different** models, then average their predicted probabilities. This is called a **simple average** ensemble — no weights to tune, no extra model on top, just the mean of each model's $P(\text{toxic})$.

The modelling journey, ending in a simple-average ensemble

Each box shows its real headline number. English Jigsaw track. Two regimes are not directly comparable.

JOURNEY (left to right)



ENSEMBLE ARCHITECTURE (simple average of three diverse base models)

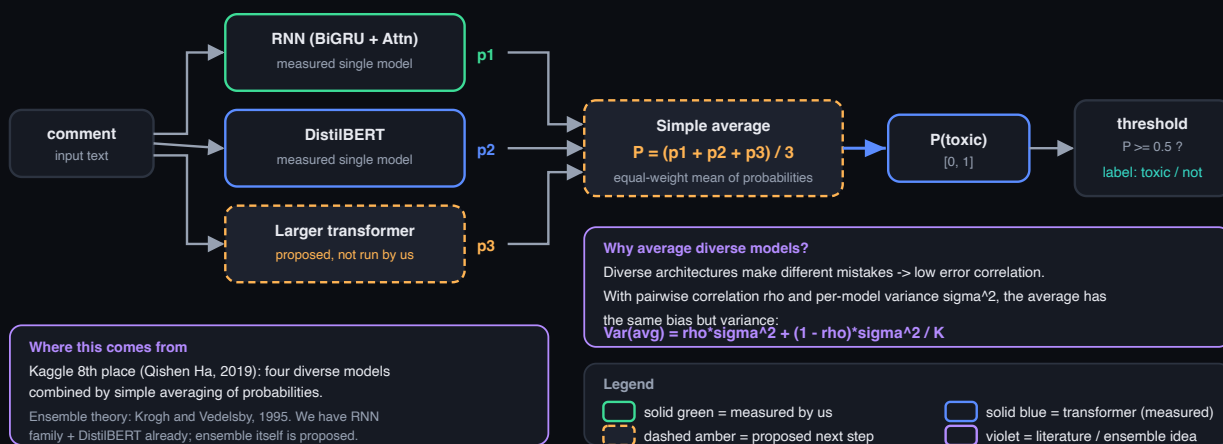


Figure 5 — the modelling journey and the simple-average ensemble.

Why it works — the maths

The result below is standard ensemble theory, not something we measured. We follow Krogh & Vedelsby (1995). LITERATURE

Setup: K models, each with the SAME bias and error variance σ^2 .
The simple average is $\bar{f} = (1/K) \times \sum f_i$.

Case 1 – independent errors:

$\text{bias}(\bar{f}) = \text{bias}(f_i)$ (unchanged)

$\text{Var}(\bar{f}) = \sigma^2 / K \rightarrow$ variance shrinks, bias does not grow.

Case 2 – errors with pairwise correlation ρ :

$\text{Var}(\bar{f}) = \rho\sigma^2 + (1 - \rho)\sigma^2 / K$

As $K \rightarrow \infty$, $\text{Var}(\bar{f}) \rightarrow \rho\sigma^2$.

$\Rightarrow$ the floor is set by ρ . Adding more copies of one model (high ρ) buys little; the gain comes from LOW correlation, i.e. DIVERSITY.

Ambiguity decomposition (Krogh & Vedelsby):

ensemble error = (mean individual error) – (mean ambiguity / diversity)

ambiguity $\geq 0 \Rightarrow$ ensemble error $\leq$ mean individual error.

(For a convex loss this also follows from Jensen's inequality: $\text{loss}(\text{mean of predictions}) \leq \text{mean of loss(predictions)}$.)

Takeaway. Averaging cuts variance without adding bias, and the benefit scales with how *uncorrelated* the models' errors are. So you average **diverse architectures** — a transformer, a larger transformer, and an RNN — not several copies of the same model.

Evidence — what the Jigsaw competition actually did

The same Jigsaw "Unintended Bias" task we used for the English track was a Kaggle competition, so we can look at what the top teams did. [LITERATURE](#)

- Qishen Ha's **8th-place** solution was titled "8th Place Solution (4 models simple avg)" — literally **four diverse models combined by simple averaging** of predicted probabilities. (We have the public title and competition write-ups; we do not have the four exact architectures, so we say "four diverse models" and do not name them.)
- The competition consensus was that a fine-tuned **BERT was the best single model**, but **every top team ensembled diverse models** — BERT-base, BERT-large, GPT-2, and LSTM-based models — by averaging or weighted-averaging (e.g. the 2nd-place "power 3.5 weighted sum"). No top team relied on a single model.
- To handle the bias metric and the imbalance, teams also used **sample weighting** (up-weight toxic + identity examples; target weight $\times \log(\text{annotator_count} + 2)$) and **negative down-sampling** (drop 50% of negatives after epoch 1).

Multi-sample dropout — a mini-ensemble inside one model

Inoue (2019, arXiv:1905.09788) gives a cheaper, related trick that lives *inside* a single model.

[LITERATURE](#)

- Within ONE training iteration, draw **k different dropout masks** and push the input through each. The duplicated layers **share weights**, so there are no extra parameters.
- Average the per-sample losses across the k masks and back-propagate that.
- Effect: faster convergence and better generalisation at almost no extra cost — an **intra-model mini-ensemble**. (We describe the mechanism only; we do not quote the paper's exact error-reduction numbers, which we have not verified.)

Our concrete plan

We already have two of the three ingredients: the **RNN family** (Stage 1) and the **fine-tuned DistilBERT** (Stage 3). The plan is to add a **larger transformer** (Stage 2's proposed step) and then average the three models' $P(\text{toxic})$. **PROPOSED NEXT STEP**

These are our real single-model starting points, kept in their own regimes (they are not directly comparable):

Model (already have)	Regime	Best single-model score	Tag
BiGRU-Conv1D + Attention (best RNN)	A — RNN-family, bias-aware AUC	Bias-AUC 0.9190	MEASURED BY US
DistilBERT fine-tuned (3-epoch saved model)	B — balanced 20k val	Acc 0.8851 · F1 0.887 · ROC-AUC 0.9515	MEASURED BY US
Larger transformer (e.g. BERT-base / BERT-large)	—	not measured	PROPOSED NEXT STEP

What we expect from the maths above plus the proven Kaggle result: averaging three diverse, low-correlation models should lower variance and reduce error below the best single model, with the same bias. That is the prediction **PROPOSED NEXT STEP** — the theory and the literature point to a gain, and our single-model numbers are the honest baseline we would measure against.

Honest caveat. We are **not** reporting a measured ensemble score. No ensemble has been run yet, so there is no number to give — that is the next experiment. We also have **no public per-model score from Qishen Ha's solution** to cite. Everything in this section is either standard theory or published competition practice, clearly labelled; the only measured numbers here are our existing single-model results.

The final model — what it does and where it stops

This is the saved English model: a fine-tuned DistilBERT classifier, served end to end. It is one model behind one API, used by three clients. Below is an honest split of what it can do and what it cannot.

What it can do

Contextual transformer

Six transformer blocks read the whole sentence at once, so the same word is read differently depending on its context — unlike the fixed word vectors used by the RNNs.

Measured accuracy

On the balanced 20k validation set: accuracy **0.8851**, F1 (toxic) **0.887**, ROC-AUC **0.9515**.

MEASURED BY US

Catches most toxic comments

Toxic recall **0.899** — it misses about one in ten toxic comments, far better than the simple high-accuracy models that miss over half.

MEASURED BY US

Served end to end

A FastAPI service runs the model, with a fallback chain (real model → LLM via gateway → keyword heuristic) so a live demo never hard-crashes.

One contract, three clients

A React website, a Chrome extension (X / YouTube / Instagram), and an Expo mobile shell all speak the same API contract.

Explanations + a tunable threshold

Per-category scores and word-level explanations (occlusion), plus a decision threshold a moderator can move to trade precision against recall.

Where it stops

- **English-only and uncased.** The tokenizer lowercases input, so it loses signal from SHOUTING in all caps. **MEASURED BY US**
- **Short inputs only.** Hard cap of **128** tokens; longer comments are truncated. **MEASURED BY US**
- **Trained on a subset, not the full corpus.** A balanced down-sample of **100,000** comments (50k per class), not all **1,804,871**. **MEASURED BY US**
- **Binary head only.** It outputs toxic / non-toxic; it does not name the toxicity type from this head.
- **Mild overfitting past one epoch.** Three epochs lowered training loss but raised validation loss (**0.537 → 0.818**) with no accuracy gain, so ~1 epoch was near-optimal for this subset. **MEASURED BY US**
- **Identity bias not yet mitigated.** The dataset's identity columns make bias measurable, but we have **not measured** or corrected it here.
- **It is a single model.** One model carries single-model variance and correlated errors — exactly what the averaging step (Stage 4) is meant to cut. **PROPOSED NEXT STEP**

Takeaway. A solid, honest single classifier — 0.8851 accuracy, 0.887 F1, 0.9515 ROC-AUC, ~0.90 toxic recall — that works end to end across three clients. Its main open weaknesses are English-only / uncased input, unmitigated identity bias, and being a single model.

Honest caveat. These numbers are for the English DistilBERT track only. They are *not* a baseline for the Hindi RAVEN-X work, which is a different language, task, and label scheme.

RAVEN-X — our original contribution (Hindi, cross-script rationale transfer)

Why we re-scoped the project

Our supervisor, Dr. Arpita Dutta, is second author of the survey "A Survey on Automatic Online Hate Speech Detection in Low-Resource Languages" (arXiv:2411.19017). That survey points to low-resource and code-mixed Hindi-English as the open gap. Building another English Jigsaw model is a reproduction, not a contribution. So she asked us to propose our own method in low-resource Hindi instead. RAVEN-X is that method.

Data truth — what we actually measured on disk

We checked the HASOC-2019 Hindi file ourselves (4,665 rows). It is native-script Hindi, not romanized Hinglish: **MEASURED BY US** 82.5% of script characters are Devanagari; 74.9% of posts are Devanagari-dominant and only 7.3% are Latin-dominant. This matters for the method: a Devanagari slur and its English equivalent share **zero** subword tokens. So any transfer from English to Hindi cannot ride on shared spelling. It has to pass through MuRIL's shared meaning space, purely as representation.

The official gold test set is blind, so we cannot use it. We froze a stratified split with seed=42 and report on it: **TRAIN 3,265 / VAL 699 / TEST 701**.

The method

One shared **MuRIL-base** encoder (12 layers, 768-d, ~236M params) feeds two heads:

CLASSIFY head

Toxic / not toxic. Supervised on Hindi HASOC and English data together.

RATIONALE head

Which words are toxic. 768→256→1 per word. Supervised **only** on English HateXplain — never on Hindi.

The two heads are tied together by a **per-head-masked multi-task loss**: each loss term applies only where its label exists, so we never train on a missing label. On top of that we add a **label-free faithfulness self-objective** on Hindi — comprehensiveness and sufficiency, measured by masking words at the attention level and re-running the full encoder. Each batch of 16 is mixed: 8 HASOC-Hindi + 8 HateXplain-English. AdamW, discriminative learning rates (2e-5 encoder / 1e-4 heads), max_len 128, fp16, on a T4 GPU.

RAVEN-X architecture

One shared encoder, two heads. Cross-script rationale transfer for Hindi.

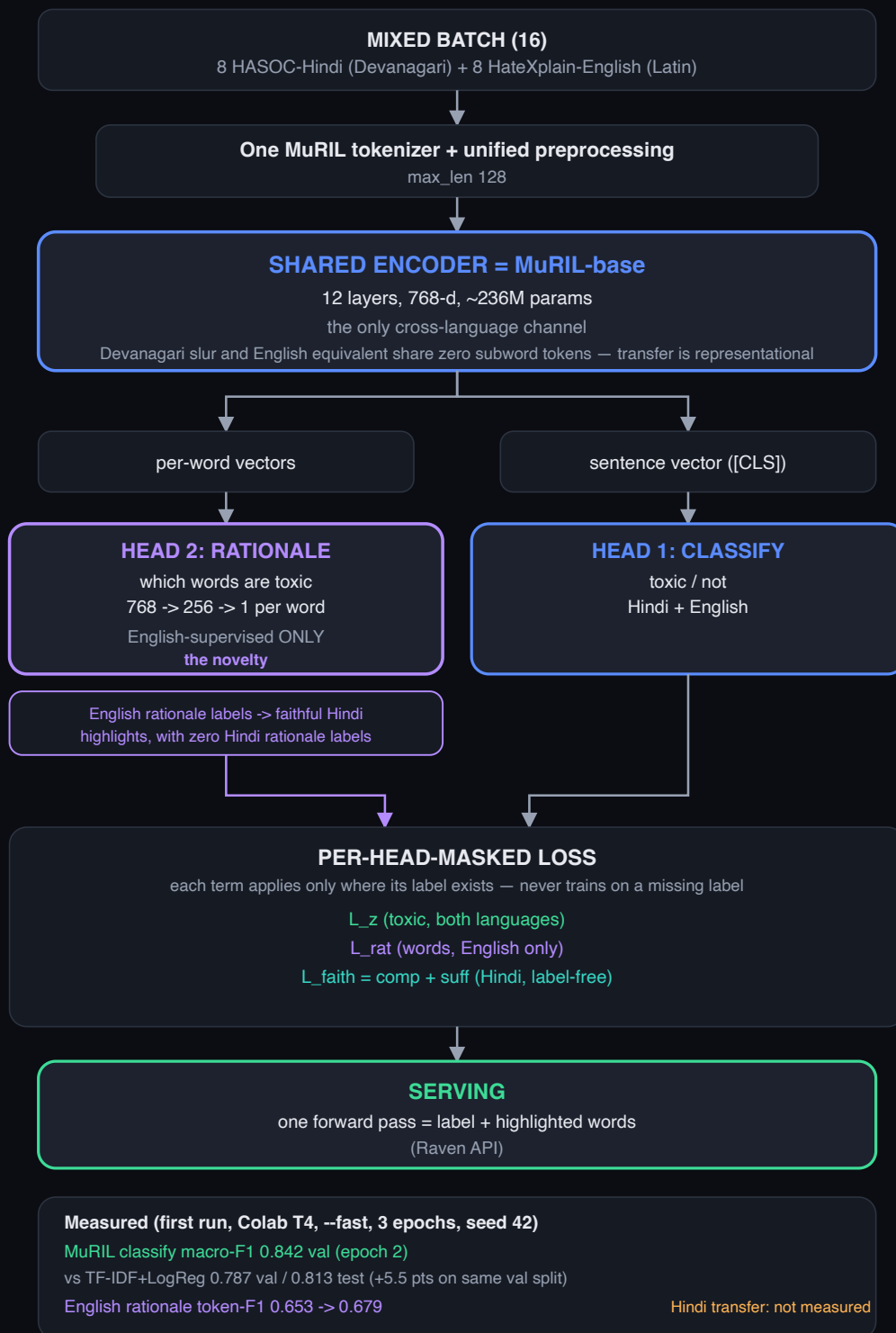


Figure 6 — RAVEN-X: one MuRIL encoder, two heads, a masked multi-task loss.

Results from the first run

Colab T4, fast mode, 3 epochs, single seed (42). All numbers below are **MEASURED BY US** on the frozen split.

Model	Macro-F1 (val)	Macro-F1 (test)	Note
TF-IDF (word+char) + LogReg baseline	0.787	0.813	classical baseline
MuRIL (RAVEN-X), epoch 2, faithfulness off	0.842	pending	+5.5 pts over baseline on the same val split

Other measured signals	Start	End
English rationale token-F1 (HateXplain)	0.653	0.679
Faithfulness trade-off (epoch 3): macro-F1 cost	0.842	0.802
Faithfulness trade-off (epoch 3): token-F1 gain	0.653	0.679

Turning the Hindi faithfulness objective on (epoch 3) traded classification macro-F1 (0.842 → 0.802) for a gain in rationale token-F1 (0.653 → 0.679). That is the accuracy-versus-explainability tension the method pre-registered, and we measured it rather than assumed it.

Why this is novel — and the honest negative-result framing

The novelty is the conjunction of three things at once: cross-script (English → Devanagari Hindi), at the rationale level (word-by-word, not just a label), with zero target-language rationale labels. We test whether explanation faithfulness survives the script boundary with a label-free deletion test we built, not an assumption. If the answer turns out to be no — that transfer does not survive crossing into Devanagari — that is still a rigorous negative result the field does not currently have, and reporting it honestly is itself a contribution.

Takeaway. RAVEN-X is one MuRIL encoder with two heads and a masked multi-task loss. On our frozen split it reached macro-F1 0.842 on validation, +5.5 points over the TF-IDF baseline (0.787), and we directly measured the accuracy-versus-explainability trade-off the design predicted.

Honest caveat. This is a fast single-seed run; MuRIL validation was capped at 400 examples by the eval loop and the clean MuRIL test number is still pending. The headline result — whether faithful explanations transfer across script into Devanagari — is the pending Week-4 measurement, not yet done. And the English DistilBERT 0.885 is **not** a baseline here: it is a different language, task, label scheme, and class balance, so comparing the two would be a category error. It served only as motivation.

What we need next

Here is the honest state of the work and the small set of things left to finish. None of the numbers below are claimed as results yet. Each item is a planned step, with the real single-model numbers we already have as the starting point.

English track

- **Train and average the ensemble.**
PROPOSED NEXT STEP Actually train three diverse models — our fine-tuned DistilBERT, a larger transformer (e.g. BERT-base), and an RNN-family model — and combine them by simple averaging of predicted probabilities. Then *measure* the gain. Our starting point is the real DistilBERT 3-epoch number: accuracy **0.8851**, F1 (toxic) **0.887**, ROC-AUC **0.9515**. The expected benefit comes from low error correlation across diverse architectures, following the Qishen Ha 8th-place "4 models simple avg" result and the standard variance-reduction theory; we will report only the measured ensemble score, not a guessed one.
- **Add multi-sample dropout.**
PROPOSED NEXT STEP Within one training step, draw k dropout masks and average the per-sample losses (Inoue, 2019). It is an intra-model mini-ensemble at almost no extra cost. We will measure whether it improves convergence and validation loss on our split.
- **Add bias-aware sample weighting.**
PROPOSED NEXT STEP Up-weight toxic and identity examples (target weight scaled by log of annotator count), as the top Jigsaw teams did, and report the Jigsaw bias-aware AUC alongside accuracy.
- **Report on one common split.**
PROPOSED NEXT STEP Right now the RNN family (large imbalanced split, bias-aware AUC) and DistilBERT (balanced 20k val) are not directly comparable. We will re-evaluate the models on one shared split so the numbers can sit in one table.

RAVEN-X track

- **Full multi-seed run.** **PROPOSED NEXT STEP**
 The current MuRIL number (val macro-F1 **0.842**, epoch 2) is a single fast seed-42 run with the val loop capped at 400 examples. We will run 3–5 seeds and report mean $\pm$ std on the full val set.
- **Script-stratified faithfulness on Devanagari.** **PROPOSED NEXT STEP** This is the headline that is *not yet measured*. We will measure comprehensiveness and sufficiency of the rationales on the native Devanagari posts (82.5% of script characters are Devanagari), since the rationale head is supervised only on English. That tests whether explanation transfers across scripts with zero shared subwords.
- **Clean held-out test number.**
PROPOSED NEXT STEP Report MuRIL on the frozen test split (701 rows). The TF-IDF baseline is macro-F1 **0.813** test; the MuRIL test number is still pending.
- **Geometry probe.** **PROPOSED NEXT STEP**
 Check, in MuRIL's shared embedding space, whether a Devanagari slur sits near its English equivalent — evidence for why cross-script transfer can work at all.

Resources we need

- **A small amount of GPU time.** **PROPOSED NEXT STEP** A free T4 is enough for the RAVEN-X multi-seed and faithfulness runs (the first run was on a Colab T4). The English ensemble plus the larger

transformer (BERT-base) needs a bit more compute than a single T4.

- **Continued access to the gated Hindi datasets.** **PROPOSED NEXT STEP** We need ongoing access to HASOC-2019 Hindi (4,665 rows) and HateXplain so we can finish the multi-seed and cross-script evaluation.

Takeaway. The remaining work is finishing and measuring, not redesigning. For English: train and average the ensemble and report a gain over our real DistilBERT numbers. For RAVEN-X: multi-seed runs, the Devanagari faithfulness headline, a clean test number, and the geometry probe. The cost is modest — mostly a little GPU time and continued dataset access.

Honest caveat. Every figure above that is tagged as a next step is not a result. The only measured numbers here are our existing single-model scores (DistilBERT 0.8851 accuracy; MuRIL 0.842 val macro-F1; TF-IDF 0.813 test macro-F1). The ensemble gain, the multi-seed mean $\pm$ std, and the Devanagari faithfulness transfer are all still to be measured.

REFERENCES

What we built on

1. **Jigsaw Unintended Bias in Toxicity Classification.** Kaggle / Conversation AI, 2019. Our dataset.
2. **Qishen Ha.** "8th Place Solution (4 models simple avg)." Jigsaw competition write-up, Kaggle, 2019. The simple-average ensemble we build the Stage 4 argument on.
3. **H. Inoue.** "Multi-Sample Dropout for Accelerated Training and Better Generalization." arXiv:1905.09788, 2019.
4. **V. Sanh, L. Debut, J. Chaumond, T. Wolf.** "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter." 2019. The backbone we fine-tuned.
5. **A. Vaswani et al.** "Attention Is All You Need." NeurIPS, 2017. The transformer / self-attention.
6. **G. Hinton, O. Vinyals, J. Dean.** "Distilling the Knowledge in a Neural Network." 2015. What knowledge distillation actually is.
7. **S. Hochreiter, J. Schmidhuber.** "Long Short-Term Memory." Neural Computation, 1997. The LSTM.
8. **A. Krogh, J. Vedelsby.** "Neural Network Ensembles, Cross Validation, and Active Learning." NeurIPS, 1995. The ambiguity decomposition behind ensembling.
9. **S. Khanuja et al.** "MuRIL: Multilingual Representations for Indian Languages." 2021. The RAVEN-X backbone.
10. **B. Mathew et al.** "HateXplain: A Benchmark Dataset for Explainable Hate Speech Detection." AAAI, 2021. The English word-level rationales.
11. **S. Das, A. Dutta, K. Roy, A. Mondal, A. Mukhopadhyay.** "A Survey on Automatic Online Hate Speech Detection in Low-Resource Languages." arXiv:2411.19017, 2024. The supervisor's survey that set the direction.

A note on honesty. The RNN/classical results and the DistilBERT results were measured under two different evaluation set-ups and are not directly comparable; the larger-transformer and the ensemble are reasoning plus published results, not numbers we have run ourselves yet; and the English DistilBERT score is not a baseline for the Hindi RAVEN-X work. These distinctions are marked throughout with **MEASURED** **LITERATURE** **PROPOSED** tags.